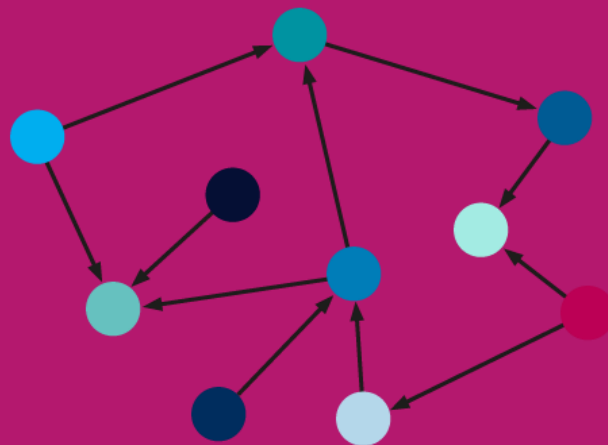
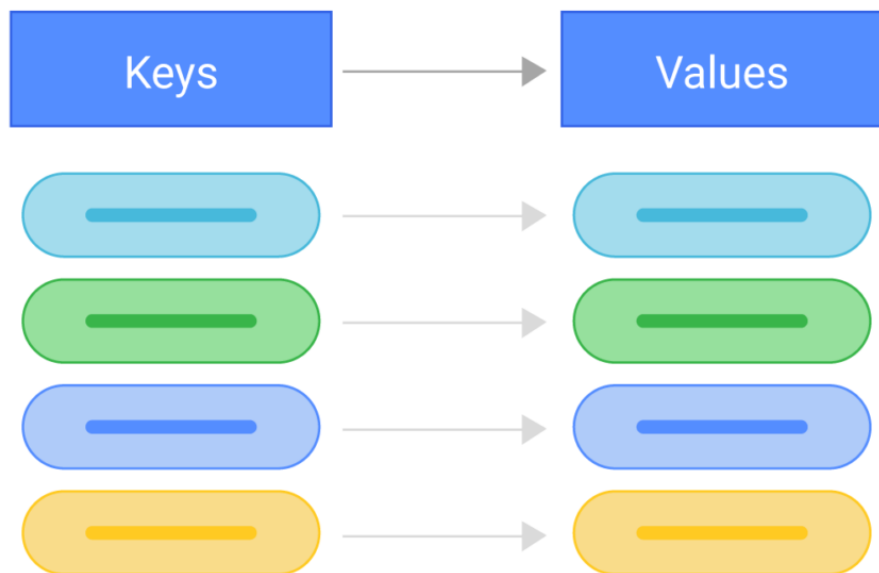




База даних типу ключ-значення

Жишкович Андрій Володимирович

База даних типу ключ-значення



Бази даних типу ключ-значення є одними з найпростіших і найшвидших систем зберігання даних.

Вони використовують просту, але ефективну модель зберігання, де кожен запис складається з унікального ключа і відповідного значення.

Сховища ключ-значення є простими способами зберігання неструктурованих даних для швидкого отримання потім. Кожен елемент інформації зберігається як пара "ключ-значення".

Ключ може бути синтетичним або автоматично згенерованим, тоді як значення можуть бути String, JSON, BLOB (бінарний великий об'єкт) тощо.

В основному тип ключ-значення використовує хеш-таблицю, в якій існує унікальний ключ та вказівник на певний елемент даних.

База даних типу ключ-значення

Phone directory

Key	Value
Paul	(091) 9786453778
Greg	(091) 9686154559
Marco	(091) 9868564334

MAC table

Key	Value
10.94.214.172	3c:22:fb:86:c1:b1
10.94.214.173	00:0a:95:9d:68:16
10.94.214.174	3c:1b:fb:45:c4:b1

Ключ і значення можуть бути будь-чим.

Підходять для додатків, які потребують зберігання та швидкого доступу до даних, забезпечуючи високопродуктивні операції читання.

Сховища типу "ключ-значення" цінні завдяки аспектам доступності та розділення, але їм бракує послідовності (Consistency). Тобто користувач може не миті побачити старі дані. Часто жертвують послідовністю заради доступності і розділення.

Часто розроблені для горизонтального масштабування.

Використання баз даних типу ключ-значення

Бази даних типу "ключ-значення" чудово підходить для зберігання великих обсягів даних для простого запиту. Деякі популярні випадки використання (у випадку Redis, до прикладу)



1

Caching

Faster performance of web applications

Кешування

Зберігання часто використовуваних даних для швидкого доступу та підвищення продуктивності.

2

Session Management

Storing and recalling user information quickly

Управління сесіями

Швидке зберігання і відновлення інформації про користувачів для підтримки активних сесій.

3

Leaderboards

Real-time, high-availability data slicing

Таблиці лідерів

Зберігання та оновлення даних про лідерів у реальному часі.

4

Inventory

Manages complex real-time inventory

Інвентаризація

Управління складними системами інвентарю в реальному часі.

5

Fraud Mitigation

Speeds fraud detection

Запобігання шахрайству

Прискорене виявлення шахрайських дій для зменшення фінансових ризиків.

6

Claims Processing

Efficient, fast processing of real-time claims

Обробка заявок

Ефективна та швидка обробка заявок клієнтів у реальному часі.

Архітектура даних

Клієнт-серверна модель:

Більшість баз даних типу ключ-значення використовують клієнт-серверну архітектуру.

Клієнт надсилає запит до сервера бази даних. Сервер отримує запит, обробляє його і виконує необхідні операції з даними. Сервер повертає результат запиту клієнту.

Розподілена система:

Деякі бази даних, такі як Amazon DynamoDB, використовують розподілену архітектуру для забезпечення високої доступності та масштабованості.

Дані розподіляються між кількома вузлами за певним алгоритмом (наприклад, хешуванням ключів). Коли клієнт надсилає запит, система визначає, на якому вузлі зберігаються потрібні дані, і перенаправляє запит до цього вузла. Вузол обробляє запит і повертає результат клієнту.

Персистентність (існування в пам'яті):

Бази даних, такі як Redis, зберігають дані в оперативній пам'яті (RAM) для забезпечення високої швидкості доступу, в той час як інші використовують HDD чи SSD.

Дані в оперативній пам'яті зберігаються тільки поки пристрій увімкнений. Після вимкнення живлення дані втрачаються.

Дані зберігаються в оперативній пам'яті, що дозволяє виконувати операції читання і запису з мінімальною затримкою. Для забезпечення стійкості даних можуть використовуватися механізми резервного копіювання або реплікації.

Структура даних

Бази даних типу ключ-значення використовують різні структури даних для зберігання інформації. Основні з них включають прості пари ключ-значення та складні типи даних.

Прості пари ключ-значення:

Основна структура даних, де кожен унікальний ключ асоціюється з одним значенням.

Приклад: Зберігання налаштувань користувача, сесій, кешування результатів запитів.

Складні типи даних:

Деякі бази даних підтримують складні типи даних, що дозволяє зберігати та маніпулювати більш структурованою інформацією.

Списки (Lists) – Послідовність елементів, упорядкованих за індексом.

Приклад: Зберігання черг завдань або повідомлень у чаті.

Множини (Sets) – Набір унікальних елементів без певного порядку.

Приклад: Зберігання унікальних тегів або категорій.

Сортовані множини (Sorted Sets) – Набір елементів, кожен з яких має унікальний бал (score), за яким елементи впорядковуються.

Приклад: Зберігання таблиць лідерів, де гравці впорядковуються за кількістю набраних балів.

Хеші (Hashes) – Колекція пар "ключ-значення" всередині одного ключа.

Приклад: Зберігання профілів користувачів, де кожен профіль має кілька атрибутів (ім'я, пошта, вік тощо).

Функціональні можливості баз даних типу ключ-значення

Основні операції

CRUD-операції: створення, читання, оновлення та видалення даних.

TTL (Time to Live): Можливість встановлення часу життя для ключів, після якого вони автоматично видаляються (наприклад, у Redis).

Масштабованість

Горизонтальне масштабування: Додавання нових вузлів для розподілу навантаження та збільшення обсягу зберігання (наприклад, у DynamoDB).

Вертикальне масштабування: Збільшення ресурсів одного вузла (наприклад, збільшення обсягу пам'яті у Redis).

Висока доступність та надійність

Реплікація: Копіювання даних на кілька вузлів для забезпечення високої доступності та стійкості до збоїв.

Автоматичне відновлення: Механізми для автоматичного відновлення даних у разі збою (наприклад, у DynamoDB).

Транзакції

Підтримка ACID-транзакцій: Деякі бази даних, такі як Oracle NoSQL, підтримують транзакції з властивостями ACID (атомарність, консистентність, ізольованість, довговічність).

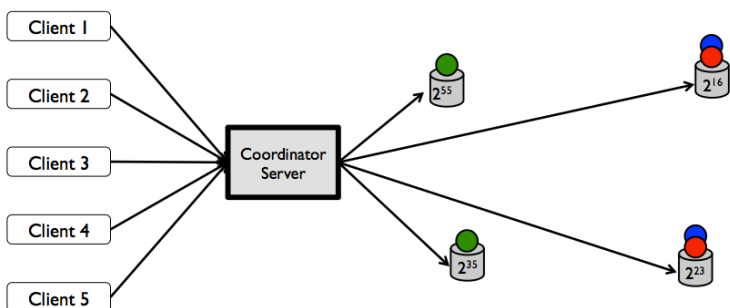
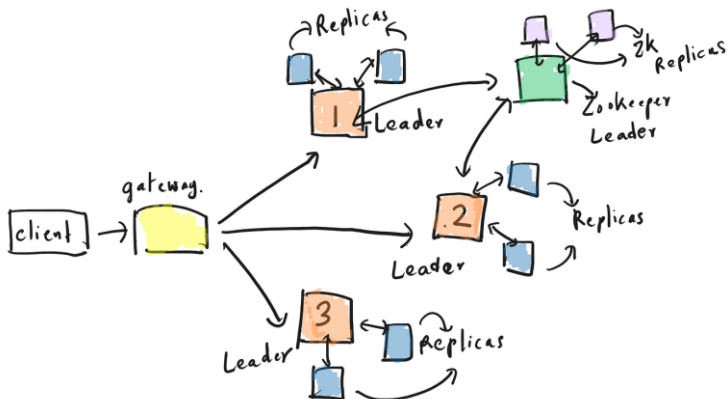
Мультикомандні транзакції: Можливість виконання кількох команд як однієї атомарної операції (наприклад, у Redis).

Розширені можливості

Публікація/підписка (Pub/Sub): Механізм для обміну повідомленнями між різними частинами системи (наприклад, у Redis).

Скрипти на Lua: Можливість виконання скриптів для складних операцій безпосередньо на сервері бази даних (наприклад, у Redis).

Розподілене сховище пар ключ-значення



Популярні рішення, такі як Amazon DynamoDB, Apache Cassandra, Redis Cluster і Riak, пропонують різноманітні можливості для реалізації цих завдань

Розподілене сховище пар ключ-значення — це тип бази даних, яка зберігає дані у вигляді пар "ключ-значення" і розподіляє ці дані між кількома вузлами (серверними машинами). Це забезпечує високу доступність, масштабованість і стійкість до збоїв.

Переваги розподілених сховищ пар ключ-значення

Масштабованість: Легко додавати нові вузли для збільшення обсягу зберігання та обробки даних.
Горизонтальне масштабування та Автоматичне розподілення даних.

Висока доступність: Дані доступні навіть у разі виходу з ладу окремих вузлів.
Реплікація та Автоматичне відновлення.

Стійкість до збоїв: Дані зберігаються на кількох вузлах, що забезпечує стійкість до збоїв окремих серверів.
Розподілене зберігання та Консенсусні алгоритми.

Гнучкість: Підтримка різних моделей узгодженості та конфігурацій.

Рейтинг баз даних типу ключ-значення

Rank			DBMS	Database Model	Score		
Oct 2025	Sep 2025	Oct 2024			Oct 2025	Sep 2025	Oct 2024
1.	1.	1.	Redis	Key-value, Multi-model	142.33	-2.84	-7.30
2.	2.	2.	Amazon DynamoDB	Multi-model	75.91	-4.37	+4.06
3.	3.	3.	Microsoft Azure Cosmos DB	Multi-model	22.71	-1.23	-1.78
4.	4.	4.	Memcached	Key-value	15.48	-0.69	-2.32
5.	5.	5.	etcd	Key-value	7.35	-0.10	+0.17
6.	6.	↑ 7.	Hazelcast	Key-value, Multi-model	5.29	+0.03	-0.27
7.	7.	↓ 6.	Aerospike	Multi-model	4.53	-0.58	-1.05
8.	8.	↑ 11.	Oracle NoSQL	Multi-model	3.35	-0.27	+0.08
9.	↑ 11.	9.	Riak KV	Key-value	3.32	0.00	-0.51
10.	↑ 12.	↑ 14.	Apache Ignite	Multi-model	2.99	-0.11	+0.05
11.	↓ 9.	↓ 8.	Ehcache	Key-value	2.99	-0.49	-1.77
12.	↑ 13.	↓ 10.	ArangoDB	Multi-model	2.91	-0.02	-0.52
13.	↓ 10.	↓ 12.	RocksDB	Key-value	2.84	-0.49	-0.20
14.	14.	↑ 17.	GemFire	Key-value, Multi-model	2.79	-0.14	+0.18
15.	15.	15.	Google Cloud Bigtable	Multi-model	2.65	-0.10	-0.26
16.	16.	↓ 13.	OrientDB	Multi-model	2.62	+0.03	-0.41
17.	17.	↓ 16.	InterSystems IRIS	Multi-model	2.20	-0.07	-0.69
18.	18.	18.	LevelDB	Key-value	2.00	-0.10	-0.39
19.	↑ 20.	↑ 20.	LMDB	Key-value	1.95	-0.06	+0.03
20.	↓ 19.	↓ 19.	Infinispan	Key-value	1.94	-0.10	-0.38
21.	↑ 22.		Valkey	Key-value	1.80	+0.06	
22.	↓ 21.	22.	Oracle Coherence	Key-value	1.75	-0.04	-0.09
23.	23.	23.	Amazon SimpleDB	Key-value	1.70	-0.01	-0.12
24.	24.	↓ 21.	Oracle Berkeley DB	Multi-model	1.64	-0.01	-0.22
25.	25.	↑ 26.	Geode	Key-value	1.38	-0.02	-0.06
26.	26.	↓ 25.	GridGain	Multi-model	1.35	-0.04	-0.14
27.	27.	↓ 24.	Tarantool	Multi-model	1.31	-0.06	-0.27
28.	28.	↓ 27.	FoundationDB	Multi-model	0.94	-0.03	-0.05
29.	29.	↓ 28.	GT.M	Key-value	0.92	+0.02	-0.05
30.	↑ 31.	↓ 29.	NCache	Key-value, Multi-model	0.83	+0.02	-0.10
31.	↓ 30.	↓ 30.	ZODR	Key-value	0.80	-0.03	-0.01

Популярні ключ-значення бази даних

Redis: Найпопулярніша key-value база даних, широко використовується як кеш чи швидке сховище даних.

Amazon DynamoDB: Серверна, керована база даних від AWS з підтримкою key-value та інших моделей даних.

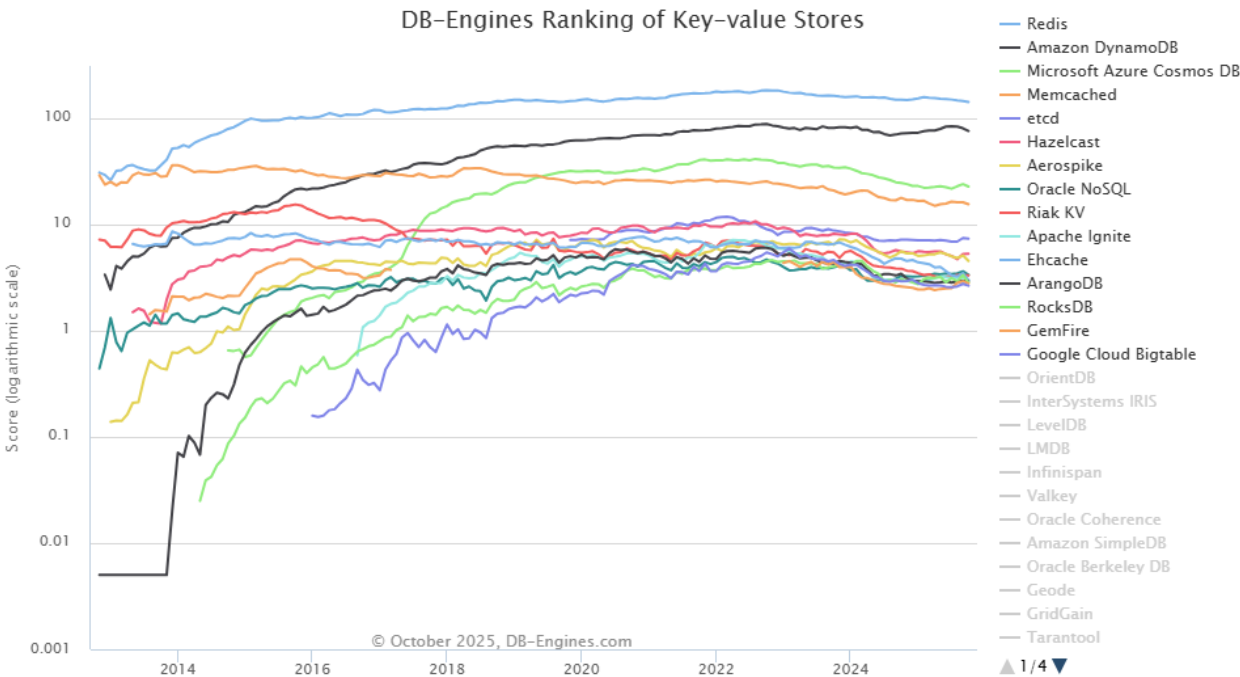
Microsoft Azure Cosmos DB: Глобально розподілена мульти-модельна база даних, що серед іншого підтримує key-value підхід.

Memcached: Легке, in-memory key-value сховище, зазвичай використовується для кешування.

etcd: Розподілена key-value база даних з консенсусом, часто використовуються для зберігання конфігурацій і координування сервісів.

ScyllaDB: Високопродуктивна NoSQL система, здатна працювати як key-value сховище.

Riak KV: Масштабована key-value система з акцентом на відмовостійкість.



Приклади баз даних типу ключ-значення

ORACLE®
NOSQL DATABASE



ORACLE®
BERKELEY DB



ДЯКУЮ ЗА УВАГУ!